Adversarial Precedent Memory: Hardening LLM Evaluators Through Mined Failure Constraints

Daniel Alami*
Independent Researcher
MBA Candidate, Harvard Business School

Abstract

LLM evaluators often fail in two distinct ways: they reward persuasive but structurally invalid outputs, and they apply hardening layers in ways that shift failures rather than uniformly eliminating them. We study evaluator hardening through three mechanisms: deterministic score gates, adversarial precedent memory, and an ordering ablation that applies precedent memory only after identifying the thesis crux. We benchmark four conditions on a mixed-family suite of 10 specimens (8 bad, 2 good) and a narrower claim-test-mismatch suite of 3 historical failures. Deterministic gates reduce reward-channel corruption relative to a soft judge. Adversarial precedent memory improves default evaluator utility across repeated mixed-family runs, primarily through lower false-accept and false-reject rates and higher mean good-specimen scores. A crux-first ablation improves detection on claim-test-mismatch failures but does not dominate on the broader suite, indicating that primitive-ordering gains are exploit-family-specific rather than uniformly dominant. Finally, the benchmark surfaced a blind spot in the evaluator itself: primitive conditioning caused a missed detection on a historical specimen, which led to a new architectural constraint and follow-up ablation. The resulting contribution is not autonomous self-improvement, but a reproducible human-in-the-loop methodology in which evaluator failures are converted into new evaluator constraints. The strongest evidence here concerns evaluator hardening, exploit-family-specific ordering effects, and this failure-to-constraint method; we do not extend the present experiments to claims of broad out-of-distribution superiority.

1. Introduction

When an LLM evaluates another LLM, two failure modes matter immediately. First, the judge can correctly describe a flaw in prose and still assign a passing score. Second, hardening layers that help on one exploit family can create new

*SSRN abstract ID: 6525598

blind spots on another. This paper studies both problems in the setting of zero-trust evaluation.

The central idea is simple: recursive gain comes from converting observed failures into reusable adversarial constraints. This is a form of failure-driven hardening: instead of trying to directly define what a correct evaluator must positively recognize, the system improves by systematically eliminating known families of failure. In this project, those constraints are stored as approved primitives: structured records of prior failure patterns, transfer conditions, safe harbors, and judge penalty logic. The key empirical twist is that these constraints help selectively rather than uniformly. Ordering and application strategy matter, and the benefit depends on exploit family.

This paper makes four claims:

1. **Deterministic score hardening reduces evaluator reward-channel corruption.**
2. **Adversarial precedent memory improves default evaluator utility on a mixed-family benchmark across repeated runs.**
3. **Primitive-ordering gains are exploit-family-specific rather than uniformly dominant.**
4. **The evaluation infrastructure can surface its own blind spots; converting those failures into architectural constraints is a reproducible human-in-the-loop methodology.**

2. System

The system under study is a zero-trust evaluator pipeline with four relevant layers.

2.1 Score-channel hardening

The original soft judge could describe a failure in natural language while still emitting a high numeric score. The hardened evaluator replaces direct free-form scoring with deterministic gating logic: the judge emits structured judgments, and Python computes the final score. Fatal failure classes can cap or zero the score.

2.2 Adversarial precedent memory

The primitive library stores mined failure precedents rather than facts or successful trajectories. Each primitive specifies:

- a failure family and mechanism
- scope conditions
- non-transfer cases
- a required transfer test
- firing-squad guidance

- a judge penalty condition
- a safe harbor

The primitive is attached to the **evaluator**, not the generator. Its purpose is to harden evaluation against recurrent exploit families rather than improve generation directly. Figure 1 shows the concrete schema used to store one such primitive.

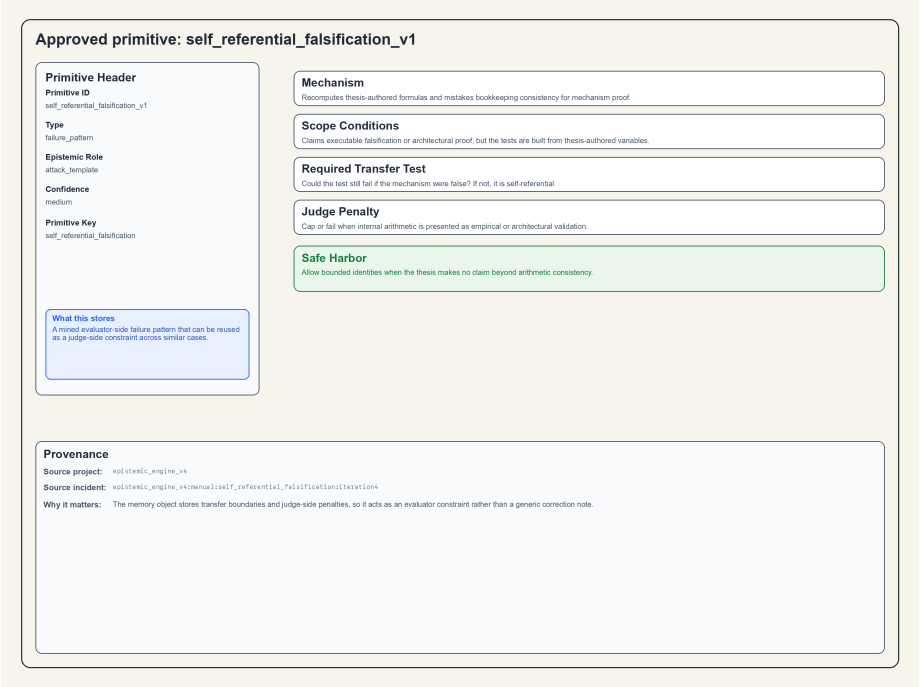

Figure 1: Example of an approved primitive (`self_referential_falsification_v1`). Each primitive stores a failure family, transfer conditions, non-transfer boundaries, and judge-side penalty logic. This makes adversarial precedent memory a structured evaluator-hardening mechanism rather than a generic correction log.

2.3 Safe harbor

Without calibration, adversarial memory overfires and kills bounded, valid local components. Safe harbor rules preserve narrow deterministic mappings that make only local claims and do not overclaim upstream truthfulness.

2.4 Evidence-Storage / Evaluator Separation

Stateful evidence accumulation is allowed outside the evaluator, but the validator remains stateless and zero-trust with respect to that state. This is the

architectural separation between evidence storage and judgment.

3. Benchmark Methodology

The benchmark design is part of the paper's contribution. Because the system under test is semantic, the benchmark must also be semantic-aware.

3.1 Main suite

The frozen main suite contains 10 specimens:

- 8 bad specimens mined from historical runs across multiple exploit families
- 2 good controls representing bounded, valid local contracts

Benchmark conditions:

- `A_baseline_soft_judge`
- `B_deterministic_gates`
- `C_gates_plus_primitives`
- `C2_gates_plus_primitives_crux_first` (used only in later ablation runs)

3.2 Claim-test-mismatch mini-suite

A second suite isolates a narrower exploit family: tests that look rigorous but prove scaffolding, tautologies, or peripheral math instead of the load-bearing claim.

Historical specimens:

- `selective_rigor_recursive_bayesian`
- `selective_rigor_simulation_god`
- `tautological_verification_central_station`

3.3 Detection metrics

We separate:

- **exploit-family detection**: did the evaluator identify the expected exploit family?
- **fatal structural detection**: did it kill the thesis for a genuinely fatal structural reason, even if the family label differed?

This distinction was necessary because exact exploit taxonomy is brittle while structural kill quality is the more stable empirical object.

3.4 Semantic adjudication

Initial keyword matching created false negatives whenever the evaluator paraphrased a correct diagnosis. We therefore added an adjudication layer for

semantic detection assessment. The adjudicator is measurement infrastructure, not part of the evaluator itself.

One architectural caution emerged during auxiliary-case triage: structural gates based on explicit test properties tend to be more stable than semantic gates that require the LLM to make a binary judgment call on an ambiguous pattern such as self-reference. We therefore treat single-run outcomes driven by those semantic binary gates as softer evidence unless they remain stable across reruns.

3.5 Why the mini-suite exists

The claim-test-mismatch mini-suite is not a second copy of the main benchmark. It isolates a specific exploit family where apparent rigor hides a failure to test the crux. This suite is what later exposed a blind spot in the primitive-conditioned evaluator itself.

4. Results I: Gates And Utility On The Main Suite

The cleanest representative run from the frozen 10-specimen main suite is `20260405_090223`, which preserves the `A -> B -> C` ladder after adding `t6_ai_inference_internal_price_floor`.

Table 1. Representative Main-Suite Run (`20260405_090223`)

Condition	N	False Accept	False Reject	Family Detection	Structural Detection	Mean Good Score
`A_baseline_soft_judge`	10	0.250	0.500	0.375	1.000	70.0
`B_deterministic_gates`	10	0.125	0.500	0.625	1.000	50.0
`C_gates_plus_primitives`	10	0.000	0.000	0.625	1.000	100.0

Interpretation:

- `B` fixes part of the score-channel problem but still falsely accepts one bad specimen.
- `C` preserves the hardening while removing false rejects in this representative run.
- the gain from `C` is best understood as **utility/calibration**, not just raw exploit detection.

Mixed-family replication summary

Across the frozen full 10-specimen main-suite reruns currently on disk (`20260405_090223`, `20260405_091143`, `20260405_092112`):

- `C` had lower false-accept rate than `B` in 1 run and tied in 2
- `C` had lower false-reject rate than `B` in 2 runs and tied in 1
- `C` had a higher mean good-specimen score in all 3 runs
- average false-accept rate: `B = 0.125`, `C = 0.083`

- average false-reject rate: `B = 0.333`, `C = 0.000`
- average mean good score: `B = 64.67`, `C = 100.0`
- average mean bad score: `B = 15.63`, `C = 10.42`
- average structural detection rate: `B = 0.958`, `C = 0.958`

The dominant remaining bad-case instability is `t2_ai_inference`: `B` falsely accepted it in all three frozen reruns, while `C` caught it once and missed it twice. That instability narrows the strength of any single-run narrative, but it does not change the overall utility pattern on the frozen benchmark.

By contrast, the newly promoted `t6_ai_inference_internal_price_floor` behaved cleanly across all three frozen runs: `A` falsely accepted it every time, while both `B` and `C` rejected it every time. That specimen is therefore the clearest stable demonstration of the `A -> B` hardening step on the frozen benchmark.

This supports a conservative result statement: **adversarial precedent memory improved the default utility of the evaluator across repeated post-calibration runs, but did not uniformly dominate on every run or metric.**

5. Results II: Ordering Ablation (`C` vs `C2`)

The ordering question arose only after the evaluator benchmark exposed a self-blind spot.

Table 2A. Discovery Run

Condition	N	False Accept	Family Detection	Structural Detection
`A_baseline_soft_judge`	3	1.000	0.333	0.667
`B_deterministic_gates`	3	0.333	0.667	0.667
`C_gates_plus_primitives`	3	0.333	0.667	1.000

This run is the blind-spot discovery. `C` caught `selective_rigor_recursive_bayesian`, which `B` missed, but `C` also missed `selective_rigor_simulation_god`, which `B` caught. That cross-over is what triggered the architectural diagnosis. The run ID is `20260404_213459`.

Table 2B. Crux-First Ablation

Condition	N	False Accept	Family Detection	Structural Detection
`A_baseline_soft_judge`	3	0.667	0.333	1.000
`B_deterministic_gates`	3	0.333	0.667	0.667
`C_gates_plus_primitives`	3	0.667	0.333	0.667
`C2_gates_plus_primitives_crux_first`	3	0.000	1.000	1.000

This run tests the targeted repair. `C2` cleans up the claim-test-mismatch suite, but the varying `C` column across the two mini-suite runs makes the stochasticity

visible rather than hiding it. The run ID is 20260404_221606.

Table 2C. Main Suite With C2 (20260404_223826, pre-freeze N=9 ablation)

Condition	N	False Accept	False Reject	Family Detection	Structural Detection	Mean Good Score
B_deterministic_gates	9	0.143	0.500	0.571	1.000	44.0
C_gates_plus_primitives	9	0.000	0.500	1.000	1.000	37.5
C2_gates_plus_primitives_crux_first	9	0.143	0.500	0.571	1.000	50.0

This is the generalization test. C2 fixes the mini-suite but does not beat C as the broader default condition. In this run it reintroduced the t2_ai_inference false accept that C had eliminated.

The resulting empirical law is narrower than "crux-first is better": **primitive-ordering gains are exploit-family-specific rather than uniformly dominant.**

6. Results III: The Evaluator Surfaced Its Own Blind Spot

The most distinctive result is not a single metric table but a process trace.

Step 1: Blind-spot discovery (20260404_213459)

On the claim-test-mismatch suite:

- selective_rigor_recursive_bayesian
  - B: passed at 100
  - C: failed at 0
- selective_rigor_simulation_god
  - B: failed at 25
  - C: passed at 100

The key point is that the score contract was not broken. The detection flags changed. Primitive conditioning helped on one specimen and hurt on another.

Step 2: Architectural diagnosis

The diagnosis was that front-loaded primitives could bias the evaluator's first reading of the crux. That produced a new constraint:

- identify the load-bearing claim first
- determine whether the test suite targets that claim
- only then inject precedent memory

Step 3: Follow-up ablation (20260404_221606)

The new C2_gates_plus_primitives_crux_first condition repaired the missed simulation_god case and cleaned up the mini-suite.

Step 4: Generalization test (20260404_223826)

C2 did not become the new default winner on the full suite. It solved one exploit-family problem and reopened another.

Figure 2 summarizes this sequence.

This is the paper's strongest methodology result:

- the evaluation infrastructure surfaced its own blind spot
- humans converted that failure into a new architectural constraint
- the new constraint was then tested directly

This is not autonomous self-improvement. It is a reproducible human-in-the-loop method for recursive evaluator hardening.

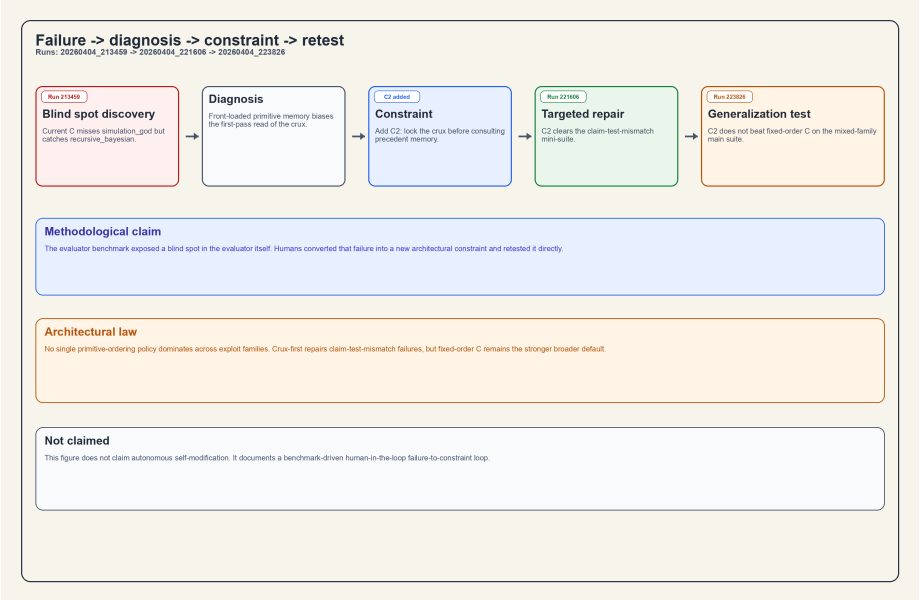

Figure 2: Benchmark-driven recursive hardening of the evaluator. A claim-test-mismatch benchmark exposed a primitive-ordering blind spot (20260404_213459), which was diagnosed as front-loaded precedent bias. That diagnosis became a new crux-first constraint (C2), which repaired the narrow exploit family (20260404_221606) but did not dominate on the broader main suite (20260404_223826).

7. Discussion

7.1 What the paper actually shows

The paper does **not** show that one hardened pipeline universally dominates. It shows:

- deterministic gates reduce reward-channel corruption
- adversarial precedent memory improves default utility on a mixed-family benchmark
- ordering benefits are exploit-family-specific
- evaluator failures can be converted into new evaluator constraints through a structured loop

7.2 Constraint vs correction

This system does not store a verbal note saying "do better next time." It stores adversarial precedents with scope, transfer, and penalty logic. That makes the memory object different from a generic correction log. A correction tries to pull the model toward a positive target; an adversarial constraint acts as a family-specific floor that prevents a known vector of ruin from being paid twice.

7.3 Why Goodhart belongs here

The score-channel problem is a reward-target problem. More subtly, the ordering-ablation result shows that hardening one layer can move failure modes rather than erase them. Goodhart is therefore a useful supporting theoretical lens [4], though not the paper's central framing.

7.4 Design implication: from fixed ordering to family-aware routing

These results establish the prerequisite for family-aware primitive routing. Fixed-ordering pipelines do not uniformly dominate across exploit families: the same ordering change that repairs claim-test-mismatch failures can reopen a different mixed-family blind spot. The next architecture should therefore move away from a single global primitive order and toward routing constraints based on the detected exploit family.

7.5 Limitations

- small benchmark (`N=10` main, `N=3` mini)
- stochastic LLM judge variance, including visible run-to-run variation in mini-suite `C` performance
- human-in-the-loop redesign rather than autonomous evaluator self-modification
- benchmark coverage is broad enough to be informative, but not broad enough for universal claims

7.6 Threats to validity

To keep the empirical claims bounded by the evidence, five threats matter most.

Small specimen count. The benchmark is intentionally narrow. False-accept and false-reject rates on `N=10` and `N=3` do not carry large-sample statistical authority and should not be read as universal laws of LLM evaluation. They function here as auditable systems traces on a curated mixed-family suite.

Judge stochasticity. The evaluator is non-deterministic, so single-run wins are not sufficient evidence of stable superiority. This is why the paper reports a post-calibration replication summary across repeated main-suite runs and treats the `C > B` result as a directional utility claim rather than a universal dominance claim.

Binary gate variance. In one auxiliary specimen, the binary gate assessment `proof_is_self_referential` flipped between runs under identical conditions, producing a `25 -> 100` score swing. This is more serious than ordinary score jitter because it changes the gate path itself. For specimens whose gaming signal lies near the LLM gate's semantic detection threshold, individual run outcomes should therefore not be treated as stable benchmark evidence. The main benchmark specimens were selected partly on repeated-run stability; auxiliary specimens did not undergo the same stability filter.

The clearest frozen-main-suite instance of this is `t2_ai_inference`. `B` missed `t2` in all three frozen reruns, which is a systematic gate failure rather than stochasticity. `C` caught `t2` once and missed it twice, which is a separate phenomenon: semantic-gate variance near the detection threshold. We therefore distinguish stable gate blind spots from run-to-run LLM detection variance when interpreting bad-case failures.

Human-in-the-loop diagnosis. The transition from benchmark anomaly to architectural constraint was performed by a human systems designer. The paper therefore demonstrates a reproducible hardening methodology, not an autonomous recursive self-improvement algorithm.

Exploit-family annotation dependence. Taxonomic labels such as `selective rigor`, `tautological verification`, or `claim-test mismatch` involve judgment. To reduce dependence on taxonomy alone, the core empirical claims rely primarily on structural kill outcomes and on transparent run-level examples rather than on exploit-family naming accuracy by itself.

Co-evolution of benchmark and system. The benchmark and evaluator were calibrated together, especially around good-control safe harbor. Early autoimmune runs are part of the design history but are separated from the post-calibration replication summary so that distinct architectural regimes are not mixed inside the same empirical claim.

7.7 Held-out stress checks and auxiliary evidence

The out-of-domain logistics specimen was useful as a held-out stress check but not as a primitive-specific wedge. In both iterations of that specimen, all three conditions rejected the thesis. This shows that the evaluator did not collapse outside the historical benchmark domains, but it does **not** isolate precedent-memory transfer advantage because the deterministic gates found conventional structural kill paths before any `domain_leakage`-specific distinction became decisive.

The auxiliary historical suite also remained auxiliary by design. One case, `central_station_hypothetical_target_laundering`, showed directional `B -> C` lift in a mixed auxiliary run (`A=59`, `B=25`, `C=0`), but it failed the stability bar for promotion when rerun in isolation (`A=59`, `B=100`, `C=100`). Another case, `central_station_mirrored_monte_carlo`, also flipped across runs. We therefore use these auxiliary results only as robustness and variance observations, not as additional core benchmark specimens.

7.8 Future work: adversarial traces as process-supervision data

The primary contribution of this paper is test-time hardening for strategic and financial reasoning, not a training-time algorithmic advance. But the ZTARE pipeline also emits structured adversarial traces: persuasive but structurally flawed trajectories that fail under execution, and surviving trajectories that withstand adversarial pressure.

Those paired traces have the right shape for future process-supervision work in domains where objective reward signals are otherwise weak. In principle, they could support supervised contrastive datasets, evaluator-training corpora, or future Process Reward Model work targeted at complex strategic reasoning.

We do not claim that the current corpus is sufficient for competitive reinforcement learning or frontier PRM training. At present, the scale is only enough to demonstrate the architectural possibility of execution-backed trace generation. The present paper's contribution remains the hardening of test-time evaluation; large-scale distillation of these traces into training-time systems is future work.

8. Related Work

The relevant contrast points are:

1. **Constraint vs correction.** Reflexion- and self-refinement-style systems store corrections or feedback traces [3, 6]. This work stores adversarial constraints.
2. **Mined vs authored.** Constitutional-style rules are hand-authored [1]. Here the constraints are mined from observed evaluator and thesis failures.
3. **Evaluator vs generator attachment.** The memory is attached to the evaluator and used to harden judgment against recurrent exploit families,

not simply to improve generation quality.

This paper also connects to:

- reward hacking / Goodhart literature [4]
- LLM-as-judge reliability literature [7, 8]
- adversarial evaluation and debate systems [2, 5]

The closest adjacent literature studies when LLM judges are reliable, how evaluator bias appears, and which judge configurations correlate best with human preference or task success [7, 8]. This paper asks a different question: how can evaluator reliability be improved through structural hardening? The contribution here is therefore not another diagnosis of judge failure, but an empirical hardening pipeline built from deterministic gates, adversarial precedent memory, and ordering ablations that expose when those hardening layers help or interfere.

References

[1] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022. doi: 10.48550/arXiv.2212.08073. URL https://arxiv.org/abs/2212.08073.

[2] Geoffrey Irving, Paul Christiano, and Dario Amodei. Ai safety via debate. *arXiv preprint arXiv:1805.00899*, 2018. doi: 10.48550/arXiv.1805.00899. URL https://arxiv.org/abs/1805.00899.

[3] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023. URL https://openreview.net/forum?id=S37hOerQLB.

[4] David Manheim and Scott Garrabrant. Categorizing variants of goodhart's law. *arXiv preprint arXiv:1803.04585*, 2019. doi: 10.48550/arXiv.1803.04585. URL https://arxiv.org/abs/1803.04585.

[5] Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. Red teaming language models with language models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 3419–3448, Abu Dhabi, United Arab Emirates, 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.225. URL https://aclanthology.org/2022.emnlp-main.225/.

[6] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R. Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. URL https://openreview.net/forum?id=vAElhFcKW6.

[7] Pat Verga, Sebastian Hofstätter, Sophia Althammer, Yixuan Su, Aleksandra Piktus, Arkady Arkhangorodsky, Minjie Xu, Naomi White, and Patrick Lewis. Replacing judges with juries: Evaluating LLM generations with a panel of diverse models. *CoRR*, abs/2404.18796, 2024. doi: 10.48550/arXiv.2404.18796. URL https://arxiv.org/abs/2404.18796.

[8] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. *CoRR*, abs/2306.05685, 2023. doi: 10.48550/arXiv.2306.05685. URL https://arxiv.org/abs/2306.05685.

Appendix A1. Main-Suite Replication Table

Only frozen full 10-specimen main-suite runs are included.

Run ID	B FA	C FA	B FR	C FR	B Mean Good	C Mean Good	B Str. Det.	C Str. Det.
20260405_090223	0.125	0.000	0.500	0.000	50.0	100.0	1.000	1.000
20260405_091143	0.125	0.125	0.500	0.000	50.0	100.0	0.875	0.875
20260405_092112	0.125	0.125	0.000	0.000	94.0	100.0	1.000	1.000
Average	**0.125**	**0.083**	**0.333**	**0.000**	**64.67**	**100.0**	**0.958**	**0.958**

Legend. FA = false accept, FR = false reject, Str. Det. = structural detection.

Footnote. Earlier `N=9` runs are retained as design history and pre-freeze ablation context, but they are excluded from this appendix because the frozen main benchmark now includes `t6_ai_inference_internal_price_floor` and is therefore a distinct evaluation regime.